Práctica 3:

Mi Agente en un Contenedor

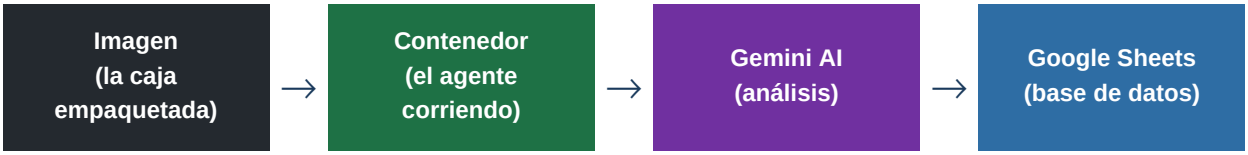
Empaquetar el Agente de Soporte con Docker

Objetivo

En la Práctica 1 el agente funcionaba en la computadora del alumno, y en la Práctica 2 se llevó a GitHub Actions. En ambos casos el agente depende de que el lugar donde corre tenga instalada la versión correcta de Python y de cada librería. En esta práctica el alumno empaquetará el agente en un **contenedor Docker**: una "caja" que incluye el script, Python y todas sus librerías, y que funciona igual en cualquier computadora o servidor que tenga Docker.

Docker es la forma estándar de desplegar programas en la industria: servicios como Railway, Render, Google Cloud Run o cualquier servidor Linux reciben una imagen de Docker y la ejecutan. Al terminar esta práctica, el agente quedará listo para correr en cualquiera de ellos, y el alumno habrá aprendido a construir imágenes, ejecutar contenedores, pasarles secretos de forma segura y revisar sus registros.

Funcionamiento del Agente en un Contenedor



La diferencia clave con las prácticas anteriores: el script no cambia ni una línea. Lo que cambia es **cómo se entrega**. En lugar de instalar Python y las librerías en cada máquina, se construye una sola vez una **imagen** con todo incluido, y a partir de ella se crean **contenedores** que ejecutan el agente. Los secretos (API Key y token) no van dentro de la imagen: se entregan al contenedor desde fuera, al momento de ejecutarlo.

Requisitos Previos

- Haber terminado la **Práctica 2**: el script debe leer la API Key con *os.getenv()*, aceptar el token por la variable *GOOGLE_TOKEN_JSON*, tener el modo *--una-vez* y contar con *requirements.txt* y *.gitignore*.
- Tener el archivo **authorized_user.json** vigente en *%APPDATA%\gspread* (el mismo que se usó para el secret GOOGLE_TOKEN_JSON en la Práctica 2).
- Windows 10 u 11 con al menos **15 GB libres** en el disco C: y la virtualización activada (se revisa en la Fase 1).

Herramienta	Para qué sirve	Costo
Docker Desktop	Construir imágenes y ejecutar contenedores en Windows	Gratuito (uso educativo y personal)
WSL 2	Subsistema de Linux de Windows; Docker Desktop lo usa por dentro	Gratuito
Visual Studio Code	Editar el Dockerfile y los archivos de configuración	Gratuito
Proyecto de la Práctica 2	Script, requirements.txt, .env y repositorio en GitHub	—

Conceptos Clave

Concepto	Qué es
Imagen	Paquete de solo lectura con todo lo necesario para ejecutar un programa: un Linux mínimo, Python, las librerías y el código. Es como la receta, o el molde.
Contenedor	Una imagen en ejecución. De una misma imagen se pueden crear muchos contenedores, y cada uno está aislado de los demás y de la computadora.
Dockerfile	Archivo de texto con las instrucciones, paso por paso, para construir una imagen.
Capa (layer)	Cada instrucción del Dockerfile genera una capa. Docker guarda las capas en caché y solo reconstruye las que cambiaron, lo que acelera mucho las reconstrucciones.
Imagen base	La imagen de la que se parte, descargada de *Docker Hub* (el repositorio público de imágenes). En esta práctica: *python:3.12-slim*.
.dockerignore	Lista de archivos que **nunca** deben copiarse a la imagen. Funciona igual que el .gitignore, pero para Docker.
Motor (engine)	El programa de Docker que construye imágenes y ejecuta contenedores. En Windows corre dentro de una pequeña máquina virtual Linux (WSL 2).

FASE 1: Instalar Docker Desktop

Paso 1.1 — Revisar los Requisitos de la Computadora

- **Espacio libre:** abrir el Explorador de archivos → *Este equipo* y revisar que el disco C: tenga al menos **15 GB libres**. Docker Desktop ocupa unos 3 GB y cada imagen agrega varios cientos de MB.
- **Virtualización:** abrir el Administrador de tareas (Ctrl+Shift+Esc) → pestaña **Rendimiento** → **CPU**. Abajo a la derecha debe decir **Virtualización: Habilitado**. Si dice *Deshabilitado*, hay que activarla en el BIOS de la computadora (la opción suele llamarse *Intel VT-x*, *Intel Virtualization Technology* o *AMD-V / SVM*).
- **WSL 2:** abrir una terminal (PowerShell) y ejecutar:

```
wsl --status
```

- Si el comando muestra *Versión predeterminada: 2*, WSL ya está listo. Si indica que WSL no está instalado, abrir PowerShell **como administrador**, ejecutar *wsl --install* y reiniciar la computadora.

Importante: No instalar Docker con el disco casi lleno. Si el disco se llena mientras Docker construye una imagen, aparecen errores como *input/output error* y después *Docker Desktop is unable to start*. Además de Docker, Windows y otros programas también empiezan a fallar sin espacio libre.

Paso 1.2 — Instalar Docker Desktop

Hay dos formas; cualquiera de las dos funciona:

- **Opción A (terminal):** en PowerShell, ejecutar el siguiente comando. Windows pedirá permisos de administrador durante la instalación; si la ventana de permisos no aparece, buscarla en la barra de tareas (suele quedar detrás de otras ventanas).

```
winget install Docker.DockerDesktop
```

- **Opción B (navegador):** descargar el instalador desde **docker.com/products/docker-desktop** y ejecutarlo con las opciones predeterminadas (dejar marcada la opción de usar WSL 2).
- Si al terminar el instalador pide cerrar sesión o reiniciar, hacerlo.

Paso 1.3 — Abrir Docker Desktop por Primera Vez

- Abrir **Docker Desktop** desde el menú Inicio.
- Aceptar los términos de servicio. Si pide iniciar sesión con una cuenta de Docker, se puede omitir (*Skip* o *Continue without signing in*); también se puede omitir la encuesta.
- Esperar a que, en la esquina inferior izquierda, aparezca **Engine running** en verde. La primera vez puede tardar uno o dos minutos.

Nota: Docker Desktop debe estar **abierto** (con el motor en verde) cada vez que se usen los comandos *docker* de esta práctica. Si se cierra, los comandos fallan con un error de conexión.

Paso 1.4 — Verificar la Instalación

Abrir una terminal **nueva** (una que se haya abierto después de instalar Docker) y ejecutar:

```
docker --version
docker run hello-world
```

El primer comando muestra la versión instalada. El segundo descarga de Docker Hub una imagen de prueba muy pequeña, crea un contenedor a partir de ella y lo ejecuta: en la terminal debe aparecer el mensaje **Hello from Docker!** seguido de una explicación de los cuatro pasos que Docker acaba de realizar. Leer esa explicación: describe exactamente lo que se hará con el agente.

Nota: Si aparece *docker no se reconoce como nombre de un cmdlet* o el error *docker-credential-desktop: executable file not found*, la terminal se abrió antes de instalar Docker y no conoce su ubicación. Cerrarla y abrir una nueva.

Nota: Si las computadoras del laboratorio no permiten instalar Docker (falta de permisos de administrador o virtualización desactivada), se puede usar **GitHub Codespaces**: en el repositorio del agente, botón **Code** → **Codespaces** → **Create codespace on main**. Se abre un Visual Studio Code en el navegador con Docker ya instalado. Los archivos secretos (.env y token.env) no están en el repositorio, así que hay que crearlos dentro del Codespace.

FASE 2: Escribir el Dockerfile

El **Dockerfile** es la receta de la imagen. Cada línea empieza con una instrucción en mayúsculas seguida de sus argumentos, y Docker las ejecuta en orden, de arriba hacia abajo. La documentación oficial está en *docs.docker.com/reference/dockerfile*.

Paso 2.1 — Crear el Archivo

- En la carpeta del proyecto (la misma donde está *agente_soporte.py*), crear un archivo llamado exactamente **Dockerfile**: con D mayúscula y **sin extensión**.

Nota: El Bloc de notas suele agregar *.txt* al guardar, y el archivo quedaría como *Dockerfile.txt*, que Docker no reconoce. Se recomienda crearlo desde Visual Studio Code.

Paso 2.2 — Contenido del Dockerfile

El Dockerfile debe contener las siguientes instrucciones, en este orden. Se recomienda agregar un comentario (línea que empieza con #) antes de cada una, explicando para qué sirve.

#	Instrucción	Qué debe hacer
1	FROM	Partir de la imagen base *python:3.12-slim*: un Linux mínimo con Python 3.12 ya instalado.
2	ENV	Definir la variable de entorno *PYTHONUNBUFFERED* con el valor *1* (ver la nota de abajo).
3	WORKDIR	Establecer */app* como carpeta de trabajo dentro del contenedor. Docker la crea si no existe.
4	COPY	Copiar **solo** el archivo *requirements.txt* a la carpeta de trabajo (el punto . como destino significa "la carpeta actual").
5	RUN	Ejecutar *pip install --no-cache-dir -r requirements.txt*. La opción *--no-cache-dir* evita guardar archivos temporales de pip y hace la imagen más pequeña.
6	COPY	Ahora sí, copiar el script *agente_soporte.py* a la carpeta de trabajo.
7	ENTRYPOINT	Indicar el comando que ejecuta el contenedor al arrancar, en formato de lista JSON: *["python", "agente_soporte.py"]*.

Concepto: ¿Por qué copiar *requirements.txt* antes que el script? Por las **capas**. Si solo cambia el script, Docker reutiliza desde la caché la capa de *pip install* (que es la más lenta) y la reconstrucción tarda segundos. Si se copiara todo junto desde el principio, cualquier cambio en el script obligaría a reinstalar todas las librerías.

Nota: PYTHONUNBUFFERED=1: normalmente Python junta los mensajes de *print()* en memoria y los escribe en bloques. En un contenedor eso provoca que los registros (Fase 5) aparezcan vacíos o con mucho retraso. Esta variable obliga a Python a mostrar cada mensaje al instante.

Concepto: Con **ENTRYPOINT**, todo lo que se escriba después del nombre de la imagen en *docker run* se agrega al final del comando. Así, la misma imagen sirve para los dos modos del agente: sin argumentos corre en modo continuo, y agregando *--una-vez* procesa y termina.

Importante: El Dockerfile **no** debe copiar el archivo .env, ni client_secrets.json, ni escribir la API Key con *ENV*. Todo lo que entra en la imagen puede leerlo cualquier persona que tenga la imagen.

Paso 2.3 — Crear el Archivo .dockerignore

Aunque el Dockerfile solo copie dos archivos, al construir la imagen Docker envía al motor toda la carpeta del proyecto (el *contexto*). El archivo **.dockerignore** excluye lo que no debe enviarse. Crearlo en la carpeta del proyecto con una línea por cada elemento:

- **.env** — contiene la API Key de Gemini.
- **token.env** — contendrá el token de Google (se crea en la Fase 4).
- **client_secrets.json** — credenciales OAuth de la aplicación.
- **.git** — el historial de Git; no lo necesita el contenedor.
- **__pycache__/** — archivos temporales de Python.

Agregar también **token.env** al archivo **.gitignore**, para que tampoco se suba a GitHub.

Paso 2.4 — Verificar la Organización de Archivos

Al terminar la Fase 2, la carpeta del proyecto debe verse así:

```
mi_agente_soporte/
  agente_soporte.py      <-- sin cambios desde la Práctica 2
  requirements.txt       <-- lo usa el Dockerfile
  Dockerfile             <-- nuevo: la receta de la imagen
  .dockerignore          <-- nuevo: lo que NO entra a la imagen
  .gitignore             <-- ahora también excluye token.env
  .env                   <-- NO entra a la imagen ni a GitHub
  client_secrets.json    <-- NO entra a la imagen ni a GitHub
  .github/workflows/     <-- el workflow de la Práctica 2 sigue igual
```

FASE 3: Construir la Imagen

Paso 3.1 — Ejecutar docker build

Con Docker Desktop abierto, en la terminal y dentro de la carpeta del proyecto, ejecutar:

```
docker build -t agente-soporte .
```

- **-t agente-soporte** le pone nombre (*tag*) a la imagen.
- El **punto final** indica que el contexto es la carpeta actual. Es fácil olvidarlo; sin él el comando falla.

La primera vez tarda uno o dos minutos: Docker descarga la imagen base y ejecuta cada instrucción del Dockerfile, mostrando un paso numerado por cada una (*[1/5] FROM*, *[2/5] WORKDIR*, etcétera). En el paso de *pip install* se ve la lista de librerías instaladas. Al final debe aparecer una línea que termina en *naming to docker.io/library/agente-soporte:latest done*.

Para confirmar que la imagen existe, ejecutar *docker images*: debe aparecer **agente-soporte** con un tamaño cercano a 270 MB.

Nota: Ejecutar *docker build* por segunda vez sin cambiar nada: debe terminar en segundos y cada paso dirá **CACHED**. Después, agregar un comentario cualquiera al script y volver a construir: solo se rehace el último COPY. Así se comprueba lo explicado sobre las capas en el Paso 2.2.

Paso 3.2 — Comprobar que la Imagen No Tiene Secretos

Antes de usar la imagen hay que verificar que no se coló ningún archivo confidencial. El siguiente comando crea un contenedor temporal que, en lugar del agente, ejecuta *ls* (listar archivos) en la carpeta */app*:

```
docker run --rm --entrypoint ls agente-soporte -A /app
```

Deben aparecer **únicamente** *agente_soporte.py* y *requirements.txt*. Si aparece .env o client_secrets.json, revisar el Dockerfile y el .dockerignore, y volver a construir la imagen.

FASE 4: Entregar los Secretos y Ejecutar una Vez

El contenedor necesita los mismos dos secretos que GitHub Actions en la Práctica 2: la API Key de Gemini y el token de Google. Igual que allá, llegarán como **variables de entorno**, y el script los leerá con *os.getenv()*. La diferencia es que aquí se entregan desde archivos de la computadora al momento de ejecutar el contenedor.

Paso 4.1 — Crear el Archivo token.env

La API Key ya está en el archivo **.env** de la Práctica 2, con el formato *NOMBRE=valor* que Docker entiende. Para el token se creará un segundo archivo con el mismo formato:

- En la carpeta del proyecto, crear un archivo llamado **token.env**.
- Escribir en él **GOOGLE_TOKEN_JSON=** (sin espacios).
- Abrir *%APPDATA%\gspread\authorized_user.json* con el Bloc de notas, copiar todo su contenido y pegarlo **inmediatamente después del signo igual, en la misma línea**. El archivo debe quedar con una sola línea, de esta forma (el valor real es mucho más largo):

```
GOOGLE_TOKEN_JSON={"token": "ya29.a0Ab...", "refresh_token": "1//0g...", ...}
```

Nota: ¿Por qué un archivo y no escribir el token directamente en el comando? En PowerShell, pasar un texto JSON con la opción *-e* de Docker rompe las comillas, y el script recibe un JSON inválido. Con *--env-file*, Docker lee cada línea tal cual, sin que la terminal la modifique.

Paso 4.2 — Ejecutar el Agente una Vez

Ejecutar el siguiente comando (en una sola línea):

```
docker run --rm --env-file .env --env-file token.env agente-soporte --una-vez
```

Parte del comando	Qué hace
docker run	Crea un contenedor a partir de una imagen y lo ejecuta.
--rm	Borra el contenedor automáticamente cuando termina (la imagen se conserva).
--env-file .env	Entrega como variables de entorno cada línea del archivo .env (GEMINI_API_KEY).
--env-file token.env	Lo mismo con token.env (GOOGLE_TOKEN_JSON).

Parte del comando	Qué hace
agente-soporte	El nombre de la imagen construida en la Fase 3.
--una-vez	Se agrega al ENTRYPOINT: el agente procesa los pendientes y termina.

La salida debe ser la misma que en la terminal o en GitHub Actions: el encabezado de ejecución, el análisis de cada fila nueva (si había alguna) y el resumen con el número de filas ya procesadas y nuevas. Al terminar, la terminal queda libre de nuevo.

Concepto: Es el mismo principio que los Secrets de GitHub: el código y la imagen se pueden compartir, y los secretos se inyectan desde fuera en el último momento. Por eso la misma imagen la puede usar cualquier alumno con **sus propios** archivos .env y token.env.

FASE 5: Ejecutar el Agente en Modo Continuo

En la Fase 4 el contenedor procesó una vez y desapareció. Ahora se dejará corriendo en segundo plano con el modo continuo de la Práctica 1 (revisando la hoja cada minuto), como lo haría en un servidor.

Paso 5.1 — Arrancar el Contenedor en Segundo Plano

Ejecutar (en una sola línea):

```
docker run -d --name agente --restart unless-stopped --env-file .env --env-file token.env agente-soporte
```

Opción	Qué hace
-d	Modo *detached*: el contenedor corre en segundo plano y la terminal queda libre.
--name agente	Le da un nombre fijo al contenedor, para referirse a él en los siguientes comandos.
--restart unless-stopped	Si el contenedor se cae o se reinicia Docker, vuelve a arrancar solo, a menos que el alumno lo haya detenido a propósito.
(sin --una-vez)	El agente corre en modo continuo, con la librería schedule.

El comando solo imprime un identificador largo y regresa a la terminal. Para confirmar que el contenedor está vivo, ejecutar *docker ps*: debe aparecer **agente** con un estado como *Up 20 seconds*. También aparece en Docker Desktop, en la sección **Containers**.

Paso 5.2 — Revisar los Registros

Como el contenedor corre en segundo plano, sus mensajes no aparecen en la terminal. Docker los guarda y se consultan así:

```
docker logs agente
docker logs -f agente
```

- **docker logs agente** muestra todo lo que el agente ha impreso desde que arrancó: el cuadro de inicio y un bloque de resumen por cada ciclo.
- **docker logs -f agente** (*follow*) se queda escuchando y muestra cada ciclo nuevo en vivo. Se sale con Ctrl+C; eso cierra la vista de registros, **no** detiene el contenedor.
- Los mismos registros se ven en Docker Desktop: **Containers** → **agente** → **Logs**.

Paso 5.3 — Prueba de Punta a Punta

- Para que sea el contenedor quien procese la prueba, pausar temporalmente el workflow de la Práctica 2: en GitHub, pestaña **Actions** → *Agente de Soporte* → botón **"..."** → **Disable workflow**.
- Dejar abierta una terminal con *docker logs -f agente*.
- Enviar una queja nueva desde el Google Form.
- En uno o dos minutos debe aparecer en los registros la fila nueva con el nombre del cliente, el comentario, la clasificación, el sentimiento y la línea **[OK] Resultado escrito en la fila N**. Revisar el Google Sheet: las columnas D, E y F de esa fila deben estar llenas.
- Al terminar la práctica, volver a activar el workflow con **Enable workflow**.

Concepto: Si Gemini está saturado, el registro puede mostrar *[ERROR] Fila N: 503 UNAVAILABLE ... high demand*. No hay que hacer nada: como el agente solo escribe "SI" en la columna F cuando todo salió bien, la fila sigue pendiente y se **reintenta sola** en el siguiente ciclo. Diseñar un agente para que tolere fallas temporales es tan importante como que funcione.

Paso 5.4 — Administrar el Contenedor

Comandos básicos para controlar el contenedor:

Comando	Qué hace
docker stop agente	Detiene el contenedor. Con *unless-stopped*, ya no vuelve a arrancar solo.
docker start agente	Vuelve a arrancar un contenedor detenido, con la misma configuración.
docker rm agente	Borra el contenedor detenido (la imagen se conserva). *docker rm -f agente* lo detiene y lo borra en un solo paso.
docker ps -a	Lista todos los contenedores, incluidos los detenidos.

Importante: Las variables de *--env-file* se leen **solo al crear** el contenedor. Si se cambia el código del script o el contenido de token.env (por ejemplo, al renovar el token), no basta con *docker restart*: hay que reconstruir la imagen si cambió el código (*docker build*), borrar el contenedor (*docker rm -f agente*) y crearlo de nuevo con el comando del Paso 5.1.

Paso 5.5 — Subir los Archivos a GitHub

Guardar el Dockerfile y el .dockerignore en el repositorio del agente:

```
git add Dockerfile .dockerignore .gitignore
git status
```

Revisar en *git status* que **no** aparezca *token.env*. Si todo está bien:

```
git commit -m "Agregar Dockerfile del agente"
git push
```

Nota: El workflow de GitHub Actions no usa el Dockerfile, así que sigue funcionando igual que antes. Docker y GitHub Actions no se excluyen: son dos formas distintas de ejecutar el mismo agente.

Solución de Errores Comunes

Error	Causa probable	Solución
docker no se reconoce como nombre de un cmdlet	La terminal se abrió antes de instalar Docker	Cerrar la terminal y abrir una nueva; si persiste, cerrar sesión en Windows
docker-credential-desktop: executable file not found	Misma causa: la terminal no conoce la carpeta de Docker	Abrir una terminal nueva
error during connect ... dockerDesktopLinuxEngine	Docker Desktop está cerrado o el motor aún no arranca	Abrir Docker Desktop y esperar a que diga Engine running
input/output error, o Docker Desktop is unable to start	El disco C: se llenó	Liberar espacio (mínimo 15 GB), cerrar Docker Desktop y volver a abrirlo
Virtualization support not detected / error de WSL	Virtualización desactivada o WSL 2 no instalado	Revisar el Paso 1.1: activar la virtualización en el BIOS o ejecutar wsl --install
failed to read dockerfile / no such file	El archivo se llama Dockerfile.txt o no está en la carpeta actual	Renombrarlo a Dockerfile sin extensión y ejecutar el comando dentro de la carpeta del proyecto
docker build requires 1 argument	Faltó el punto final del comando	Usar docker build -t agente-soporte . (con el punto)
"Debes reemplazar GEMINI_API_KEY" dentro del contenedor	Faltó --env-file .env, o el .env no tiene la clave	Revisar el comando y el contenido del archivo .env
"No se encontró el archivo client_secrets.json" dentro del contenedor	Faltó --env-file token.env, o el archivo no empieza con GOOGLE_TOKEN_JSON=	Revisar el comando y el Paso 4.1
JSONDecodeError al leer GOOGLE_TOKEN_JSON	El token quedó en varias líneas o incompleto en token.env	Dejar todo en una sola línea, incluyendo las llaves { } del inicio y del final
The container name "/agente" is already in use	Ya existe un contenedor con ese nombre	Borrarlo con docker rm -f agente y volver a ejecutar el Paso 5.1
docker logs agente no muestra nada	Falta la variable PYTHONUNBUFFERED en el Dockerfile	Agregar la instrucción ENV del Paso 2.2, reconstruir y crear de nuevo el contenedor

Error	Causa probable	Solución
503 UNAVAILABLE ... high demand	Gemini está saturado en ese momento	Nada: la fila se reintenta sola en el siguiente ciclo
invalid_grant dentro del contenedor	El token de token.env caducó o fue revocado	Generar un token nuevo, actualizar token.env y crear de nuevo el contenedor (Paso 5.4)

Comparación: Prácticas 1, 2 y 3

Aspecto	Práctica 1 (computadora)	Práctica 2 (GitHub Actions)	Práctica 3 (Docker)
¿Dónde corre?	En la computadora del alumno	En un runner de GitHub	En cualquier máquina con Docker
¿Quién instala Python y librerías?	El alumno, con pip	El workflow, en cada ejecución	Se instalan una vez dentro de la imagen
¿Quién lo repite?	schedule + while True	El cron del workflow	schedule dentro del contenedor
API Key de Gemini	Archivo .env	Secret GEMINI_API_KEY	--env-file .env
Sesión de Google	Navegador + authorized_user.json	Secret GOOGLE_TOKEN_JSON	--env-file token.env
¿Dónde se ven los mensajes?	En la terminal	En la pestaña Actions	docker logs agente
¿Y si se cae?	Hay que volver a ejecutarlo	Se repite en el siguiente cron	--restart lo vuelve a arrancar

¿Y después? La imagen construida en esta práctica es la misma que aceptan los servicios de nube que ejecutan contenedores (Railway, Render, Google Cloud Run) o un servidor Linux propio. Para usarla allí basta con subirla a un registro de imágenes, como Docker Hub o GitHub Container Registry, y configurar en el servicio las mismas dos variables de entorno.

Entregables

- **Repositorio:** el Dockerfile y el .dockerignore subidos al repositorio del agente (Paso 5.5), al que el profesor ya tiene acceso desde la Práctica 2.
- **Captura 1:** la salida de *docker run hello-world* (Paso 1.4).
- **Captura 2:** el final de *docker build* y la imagen en *docker images* (Paso 3.1).
- **Captura 3:** la lista de archivos de la imagen, sin secretos (Paso 3.2).
- **Captura 4:** la ejecución con *--una-vez* (Paso 4.2).
- **Captura 5:** *docker logs agente* mostrando una queja procesada en la prueba de punta a punta, y el Google Sheet con esa fila clasificada (Paso 5.3).
- **Respuestas** a las preguntas de reflexión.

Preguntas de Reflexión

- Explicar con un ejemplo propio la diferencia entre una imagen y un contenedor.
- Si el .dockerignore no excluyera el archivo .env y la imagen se publicara en Docker Hub, ¿qué podría hacer otra persona con ella?
- ¿Por qué el Dockerfile copia requirements.txt antes que el script? ¿Qué se observó en el Paso 3.1 al reconstruir la imagen?
- ¿Por qué la misma imagen puede ejecutarse en modo continuo o con *--una-vez* sin construir dos imágenes distintas?
- Si el token de Google se renueva, ¿por qué no basta con *docker restart agente*?
- Para este agente, ¿cuándo convendría usar GitHub Actions y cuándo un contenedor en un servidor? Considerar costo, frecuencia de revisión y facilidad de mantenimiento.